

MODUL 4

Nama: Ryan Akeyla Novianto Widodo

Kelas: 12 IF 07

NIM : 103112400081

DASAR TEORI

Dasar Teori C++

Diambil dari internet dan dirangkum ulang oleh saya dan dibantu dengan AI.

1. Inti Program

- Syntax: Mirip bahasa Inggris, tapi huruf besar dan kecil beda arti. Setiap perintah biasanya diakhiri dengan titik koma (;).

- Komentar: Catatan di kode yang tidak dijalankan.

- Satu baris: `// Ini komentar`

- Banyak baris: `/* Ini komentar yang panjang */`

- Header: File yang berisi daftar perintah dan fungsi yang bisa dipakai. Contoh:

cpp

```
#include <iostream> // Untuk perintah input/output (cout, cin)
```

- `main()` : Fungsi utama, tempat program mulai berjalan.

cpp

```
int main() {
```

```
    // Kode program di sini
```

```
    return 0; // Menandakan program selesai dengan benar
```

```
}
```

- Namespace: Seperti "area" untuk nama-nama perintah. `std` adalah namespace standar C++.

cpp

```
using namespace std; // Supaya tidak perlu tulis std::cout, std::cin, dll.
```

- Input/Output:

- cout : Menampilkan sesuatu di layar.

- cin : Membaca input dari keyboard.

cpp

```
cout << "Halo!" << endl; // Menampilkan "Halo!"
```

```
int umur;
```

```
cin >> umur; // Membaca angka dari keyboard dan disimpan di variabel "umur"
```

2. Jenis Data

- Integer (Bilangan Bulat):

- int : Bilangan bulat (contoh: -10, 0, 25).

- short : Bilangan bulat pendek.

- long : Bilangan bulat panjang.

- Floating-Point (Bilangan Desimal):

- float : Bilangan desimal (contoh: 3.14).

- double : Bilangan desimal dengan ketelitian lebih tinggi.

- Character (Karakter):

- char : Satu huruf, angka, atau simbol (contoh: 'a', 'Z', '5').

- Boolean (Logika):

- bool : Nilai benar (true) atau salah (false).

- void : "Kosong". Dipakai untuk fungsi yang tidak menghasilkan nilai.

3. Variabel

- Deklarasi: Memberi tahu program nama dan jenis data variabel.

cpp

```
int umur; // Variabel bernama "umur" untuk menyimpan bilangan bulat
```

```
double harga; // Variabel bernama "harga" untuk menyimpan bilangan desimal
```

- Inisialisasi: Memberi nilai awal ke variabel.

cpp

```
int umur = 20; // "umur" diisi dengan nilai 20
```

```
double harga = 99.50; // "harga" diisi dengan nilai 99.50
```

4. Operator

- Aritmatika: + (tambah), - (kurang), * (kali), / (bagi), % (sisa bagi).
- Penugasan: = (mengisi nilai ke variabel).
- Perbandingan: == (sama dengan), != (tidak sama dengan), > (lebih besar), < (lebih kecil), >= (lebih besar atau sama dengan), <= (lebih kecil atau sama dengan).
- Logika: && (DAN), || (ATAU), ! (TIDAK).
- Increment/Decrement: ++ (tambah 1), -- (kurang 1).

5. Struktur Kontrol

- if : Melakukan sesuatu jika kondisi benar.

cpp

```
if (umur >= 17) {  
    cout << "Anda sudah dewasa." << endl;  
} else {  
    cout << "Anda belum dewasa." << endl;  
}
```

- switch : Memilih salah satu dari beberapa pilihan.

cpp

```
int hari = 3;  
switch (hari) {  
    case 1:  
        cout << "Senin" << endl;  
        break;  
    case 2:
```

```

        cout << "Selasa" << endl;

        break;

    case 3:

        cout << "Rabu" << endl;

        break;

    default:

        cout << "Hari lain" << endl;

}

```

- for : Melakukan sesuatu berulang kali sebanyak yang ditentukan.

cpp

```

for (int i = 0; i < 5; i++) {

    cout << i << " "; // Menampilkan angka 0 sampai 4

}

cout << endl;

```

- while : Melakukan sesuatu berulang kali selama kondisi benar.

cpp

```

int angka = 1;

while (angka <= 3) {

    cout << angka << " ";

    angka++;

}

cout << endl;

```

- do-while : Sama seperti while , tapi dilakukan minimal sekali.

cpp

```

int pilihan;

do {

    cout << "Pilih 1, 2, atau 3: ";

```

```
cin >> pilihan;  
} while (pilihan < 1 || pilihan > 3);
```

6. Fungsi

- Definisi: Sekumpulan kode yang melakukan tugas tertentu.

cpp

```
int tambah(int a, int b) {  
    return a + b;  
}
```

- Pemanggilan: Menggunakan fungsi.

cpp

```
int hasil = tambah(5, 2); // Memanggil fungsi "tambah" dengan angka 5 dan 2  
cout << "Hasil: " << hasil << endl; // Menampilkan "Hasil: 7"
```

7. Pointer

- Definisi: Variabel yang menyimpan alamat memori variabel lain. (Agak rumit, tapi penting).

cpp

```
int umur = 25;  
int *p_umur = &umur; // p_umur menyimpan alamat memori dari variabel umur
```

DASAR TEORI LINKED LIST

Singly Linked List adalah struktur data yang fleksibel dan efisien untuk banyak aplikasi. Memahami dasar teori tentang Singly Linked List sangat penting untuk memilih struktur data yang tepat untuk masalah tertentu dan untuk mengimplementasikan algoritma yang efisien.

Definisi:

- Linked List (Daftar Berantai): Struktur data linear di mana elemen-elemen data (disebut node) disusun secara sekuensial, dengan setiap node berisi data dan pointer (penunjuk) ke node berikutnya dalam urutan.

- Singly Linked List (Daftar Berantai Tunggal): Jenis linked list di mana setiap node hanya memiliki satu pointer, yaitu pointer ke node berikutnya. Ini memungkinkan traversal (penelusuran) list hanya dalam satu arah (dari awal ke akhir).

Komponen Utama:

1. Node (Simpul):

- Unit dasar dalam linked list.

- Setiap node berisi dua bagian:

- Data: Informasi yang disimpan dalam node (bisa berupa tipe data apa saja, seperti integer, string, objek, dll.).

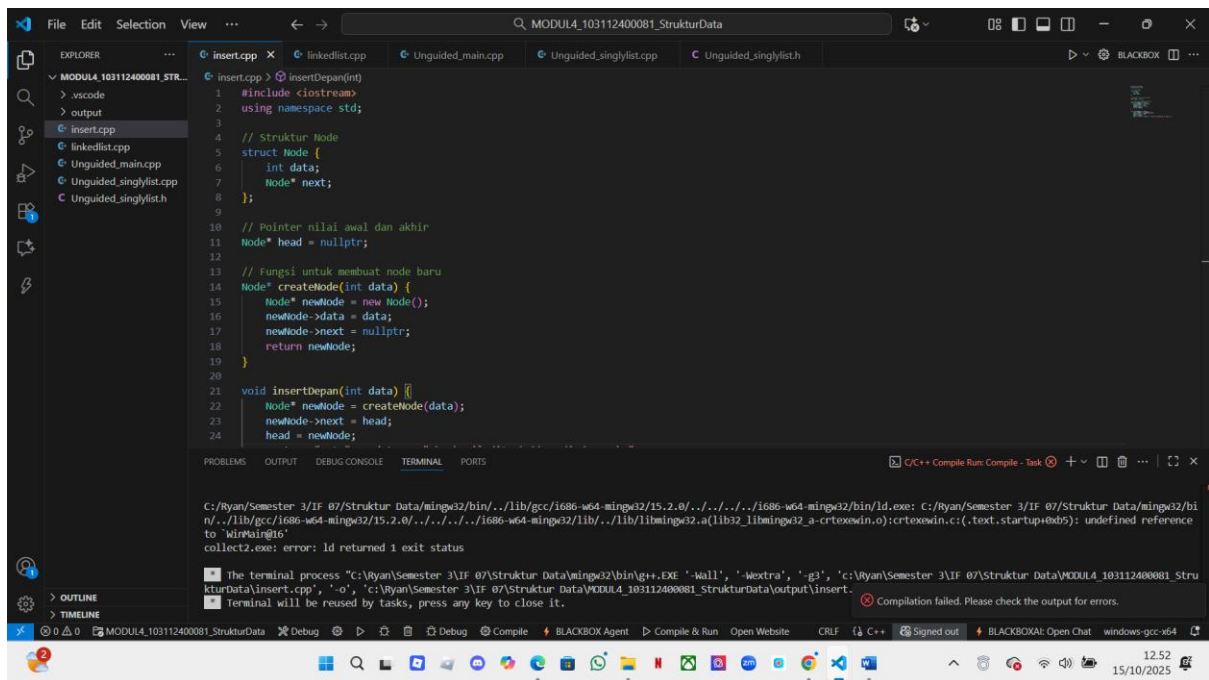
- Next (Berikutnya): Pointer yang menunjuk ke node berikutnya dalam list. Jika node ini adalah node terakhir, pointer next akan berisi nilai NULL (atau nullptr dalam C++11 ke atas).

2. Head (Kepala):

- Pointer yang menunjuk ke node pertama dalam list.

- Jika list kosong, pointer head akan berisi nilai NULL .

Guided 1



```
File Edit Selection View ... MODULA_103112400081_StrukturData
EXPLORER
  MODULA_103112400081_STR...
    .vscode
    > output
  insert.cpp
  linkedlist.cpp
  Unguided_main.cpp
  Unguided_singlylist.cpp
  Unguided_singlylist.h
  insert.cpp
    1 #include <iostream>
    2 using namespace std;
    3
    4 // Struktur Node
    5 struct Node {
    6     int data;
    7     Node* next;
    8 };
    9
    10 // Pointer nilai awal dan akhir
    11 Node* head = nullptr;
    12
    13 // Fungsi untuk membuat node baru
    14 Node* createNode(int data) {
    15     Node* newNode = new Node();
    16     newNode->data = data;
    17     newNode->next = nullptr;
    18     return newNode;
    19 }
    20
    21 void insertDepan(int data) {
    22     Node* newNode = createNode(data);
    23     newNode->next = head;
    24     head = newNode;
    25 }
    26
    27 int main() {
    28     // ...
    29 }
    30
    31 return 0;
    32 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Ryan\Semester 3\IF 07\Struktur Data\mingw32\bin\../lib/gcc/i686-w64-mingw32/15.2.0/../../../../i686-w64-mingw32/bin/ld.exe: C:\Ryan\Semester 3\IF 07\Struktur Data\mingw32\bin\../lib/gcc/i686-w64-mingw32/15.2.0/../../../../i686-w64-mingw32/lib/./lib/libmingw32.a(lib32_libmingw32_a-crtexwin.o):crtexwin.c:(.text.startup+0b5): undefined reference to 'winMain@16'

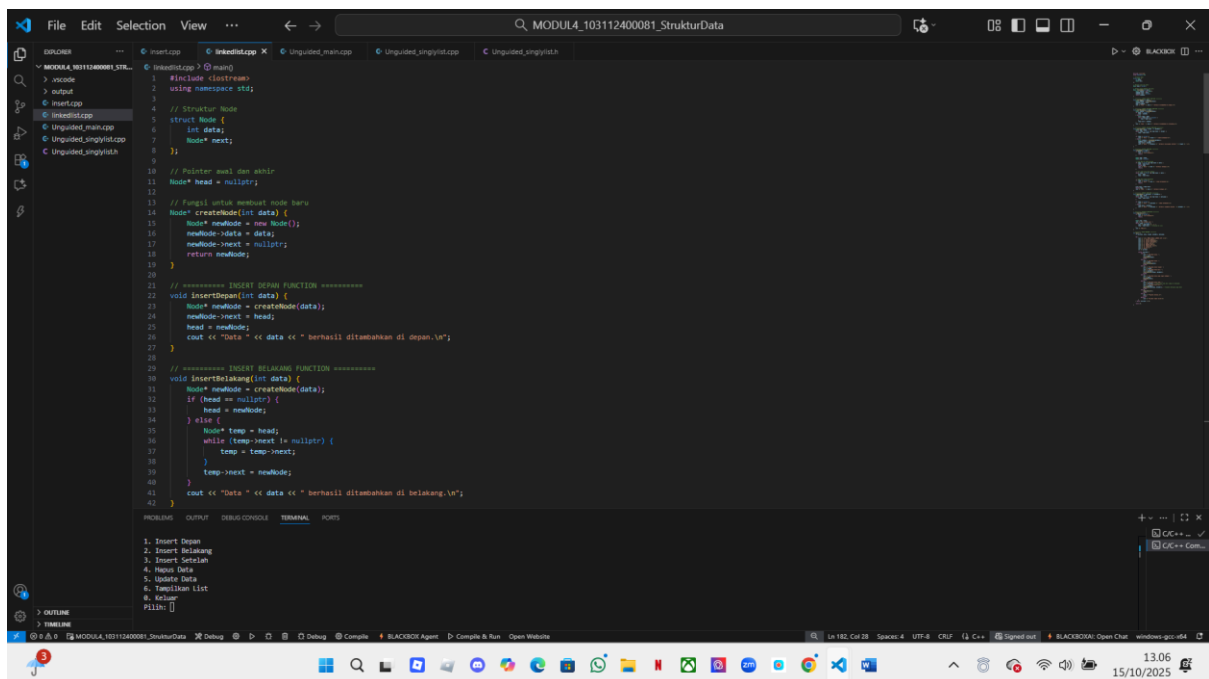
collect2.exe: error: ld returned 1 exit status

The terminal process "C:\Ryan\Semester 3\IF 07\Struktur Data\mingw32\bin\g++.exe -Wall, -Wextra, -g3, 'C:\Ryan\Semester 3\IF 07\Struktur Data\MODULA_103112400081_StrukturData\insert.cpp', '-o', 'C:\Ryan\Semester 3\IF 07\Struktur Data\MODULA_103112400081_StrukturData\output\insert.exe' Compilation failed. Please check the output for errors.

Terminal will be reused by tasks, press any key to close it.

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita mengimplementasikan ADT Singly Linked List di C++ yang mencakup operasi dasar seperti membuat list, mengalokasikan dan dealokasi memori, menyisipkan elemen di awal list, dan menampilkan isi list. Tujuannya adalah untuk mendemonstrasikan konsep struktur data linked list dan cara menggunakannya dalam program.

Guided 2



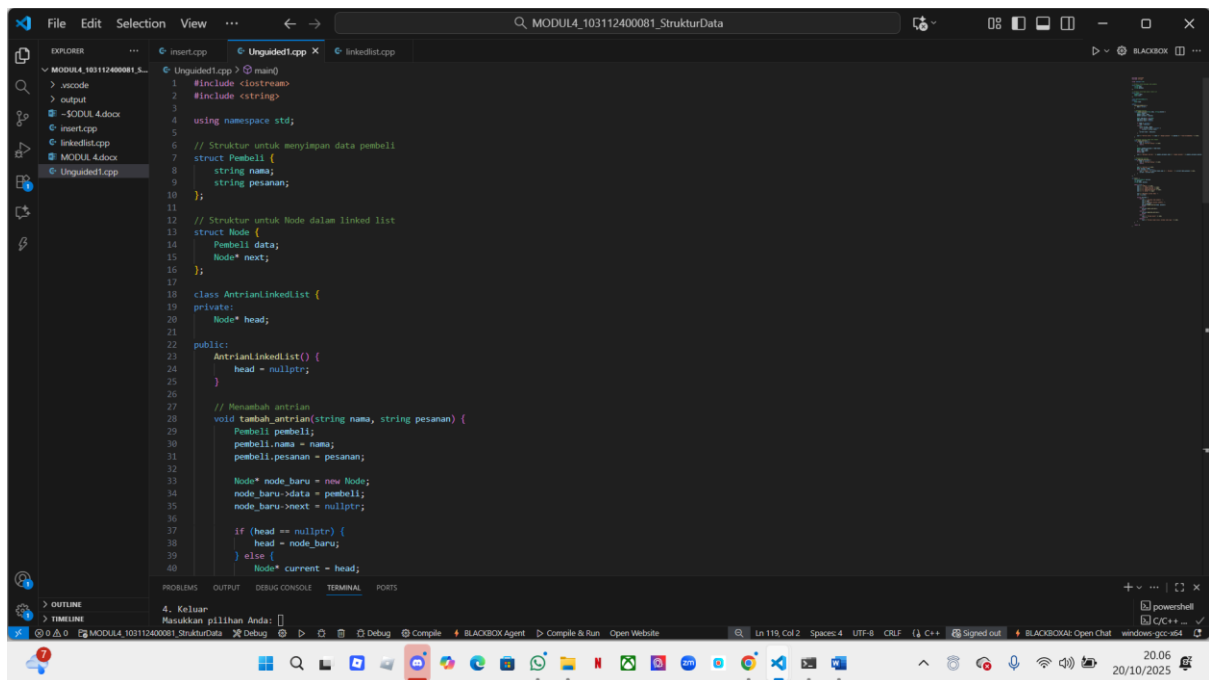
```
File Edit Selection View ... MODULA_103112400081_StrukturData
EXPLORER
  MODULA_103112400081_STR...
    .vscode
    > output
  insert.cpp
  linkedlist.cpp
  Unguided_main.cpp
  Unguided_singlylist.cpp
  Unguided_singlylist.h
  insert.cpp
    1 #include <iostream>
    2 using namespace std;
    3
    4 // Struktur Node
    5 struct Node {
    6     int data;
    7     Node* next;
    8 };
    9
    10 // Pointer awal dan akhir
    11 Node* head = nullptr;
    12
    13 // Fungsi untuk membuat node baru
    14 Node* createNode(int data) {
    15     Node* newNode = new Node();
    16     newNode->data = data;
    17     newNode->next = nullptr;
    18     return newNode;
    19 }
    20
    21 // ===== INSERT DEPAN FUNCTION =====
    22 void insertDepan(int data) {
    23     Node* newNode = createNode(data);
    24     newNode->next = head;
    25     head = newNode;
    26     cout << "Data " << data << " berhasil ditambahkan di depan.\n";
    27 }
    28
    29 // ===== INSERT BELAKANG FUNCTION =====
    30 void insertBelakang(int data) {
    31     Node* newNode = createNode(data);
    32     if (head == nullptr) {
    33         head = newNode;
    34     } else {
    35         Node* temp = head;
    36         while (temp->next != nullptr) {
    37             temp = temp->next;
    38         }
    39         temp->next = newNode;
    40     }
    41     cout << "Data " << data << " berhasil ditambahkan di belakang.\n";
    42 }
    43
    44 int main() {
    45     // ...
    46 }
    47
    48 return 0;
    49 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

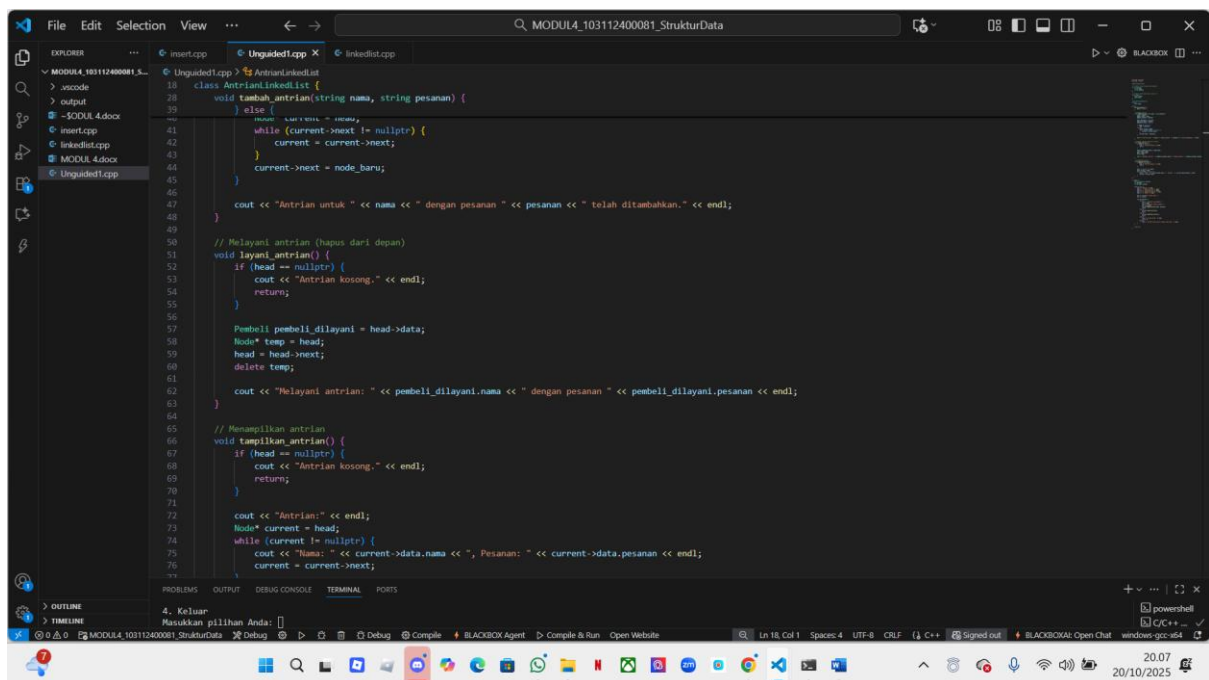
1. Insert Depan
2. Insert Belakang
3. Insert Setoran
4. Menus Data
5. Update Data
6. Tampilkan List
8. Exit
Pilih: 1

Tujuan program ini adalah membuat program dari bahasa c++ yang mana bertujuan untuk mengimplementasikan dan mendemonstrasikan operasi pada sebuah Singly Linked List (Daftar Berantai Tunggal). kode ini bertujuan untuk menyediakan alat yang memungkinkan pengguna untuk membuat, memanipulasi, dan melihat isi dari sebuah Singly Linked List melalui menu interaktif. Ini berguna untuk tujuan pembelajaran dan demonstrasi konsep linked list.

Unguided 1



```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Struktur untuk menyimpan data pembeli
6 struct Pembeli {
7     string nama;
8     string pesanan;
9 };
10
11 // Struktur untuk Node dalam linked list
12 struct Node {
13     Pembeli data;
14     Node* next;
15 };
16
17 class AntrianLinkedList {
18 private:
19     Node* head;
20 public:
21     AntrianLinkedList() {
22         head = nullptr;
23     }
24
25     // Menambah antrian
26     void tambah_antrian(string nama, string pesanan) {
27         Pembeli pembeli;
28         pembeli.nama = nama;
29         pembeli.pesanan = pesanan;
30
31         Node* node_baru = new Node;
32         node_baru->data = pembeli;
33         node_baru->next = nullptr;
34
35         if (head == nullptr) {
36             head = node_baru;
37         } else {
38             Node* current = head;
39             while (current->next != nullptr) {
40                 current = current->next;
41             }
42             current->next = node_baru;
43         }
44     }
45
46     // Menampilkan antrian
47     void tampilkan_antrian() {
48         if (head == nullptr) {
49             cout << "Antrian kosong." << endl;
50             return;
51         }
52         cout << "Antrian:" << endl;
53         Node* current = head;
54         while (current != nullptr) {
55             cout << "Nama: " << current->data.nama << ", Pesanan: " << current->data.pesanan << endl;
56             current = current->next;
57         }
58     }
59
60     // Melayani antrian (hapus dari depan)
61     void layani_antrian() {
62         if (head == nullptr) {
63             cout << "Antrian kosong." << endl;
64             return;
65         }
66         Pembeli pembeli_dilayani = head->data;
67         Node* temp = head;
68         head = head->next;
69         delete temp;
70         cout << "Melayani antrian: " << pembeli_dilayani.nama << " dengan pesanan " << pembeli_dilayani.pesanan << endl;
71     }
72
73     // Menampilkan menu
74     void tampilkan_menu() {
75         cout << "1. Menambah antrian\n";
76         cout << "2. Melayani antrian\n";
77         cout << "3. Menampilkan antrian\n";
78         cout << "4. Keluar\n";
79     }
80
81     // Memulai program
82     void mulai() {
83         tampilkan_menu();
84         int pilihan;
85         do {
86             cout << "Masukkan pilihan Anda: ";
87             while (getchar() != '\n') continue;
88             pilihan = getchar();
89             switch (pilihan) {
90                 case '1': tambah_antrian(" ", " "); break;
91                 case '2': layani_antrian(); break;
92                 case '3': tampilkan_antrian(); break;
93                 case '4': return;
94                 default: cout << "Pilihan tidak valid. Silakan pilih kembali.\n";
95             }
96             tampilkan_menu();
97         } while (pilihan != 4);
98     }
99
100     ~AntrianLinkedList() {
101         while (head != nullptr) {
102             Node* temp = head;
103             head = head->next;
104             delete temp;
105         }
106     }
107 }
```



```
1 // Melayani antrian (hapus dari depan)
2 void layani_antrian() {
3     if (head == nullptr) {
4         cout << "Antrian kosong." << endl;
5         return;
6     }
7     Pembeli pembeli_dilayani = head->data;
8     Node* temp = head;
9     head = head->next;
10    delete temp;
11    cout << "Melayani antrian: " << pembeli_dilayani.nama << " dengan pesanan " << pembeli_dilayani.pesanan << endl;
12 }
13
14 // Menampilkan menu
15 void tampilkan_menu() {
16     cout << "1. Menambah antrian\n";
17     cout << "2. Melayani antrian\n";
18     cout << "3. Menampilkan antrian\n";
19     cout << "4. Keluar\n";
20 }
21
22 // Memulai program
23 void mulai() {
24     tampilkan_menu();
25     int pilihan;
26     do {
27         cout << "Masukkan pilihan Anda: ";
28         while (getchar() != '\n') continue;
29         pilihan = getchar();
30         switch (pilihan) {
31             case '1': tambah_antrian(" ", " "); break;
32             case '2': layani_antrian(); break;
33             case '3': tampilkan_antrian(); break;
34             case '4': return;
35             default: cout << "Pilihan tidak valid. Silakan pilih kembali.\n";
36         }
37         tampilkan_menu();
38     } while (pilihan != 4);
39 }
40
41 ~AntrianLinkedList() {
42     while (head != nullptr) {
43         Node* temp = head;
44         head = head->next;
45         delete temp;
46     }
47 }
48
49 int main() {
50     AntrianLinkedList antrian;
51     antrian.mulai();
52     return 0;
53 }
```


The screenshot shows the Visual Studio Code editor with a C++ file named `Unguided1.cpp`. The code implements a linked list structure. It includes a `Node` struct with `data` and `next` pointers. A `LinkedList` class is defined with methods for adding nodes (`addNode`), displaying the list (`displayList`), and a `main` function that provides a menu for the user to interact with the list. The menu options are: 1. Tambah Antrian, 2. Layani Antrian, 3. Tampilkan Antrian, and 4. Keluar. The `main` function uses a `switch` statement to handle these options.

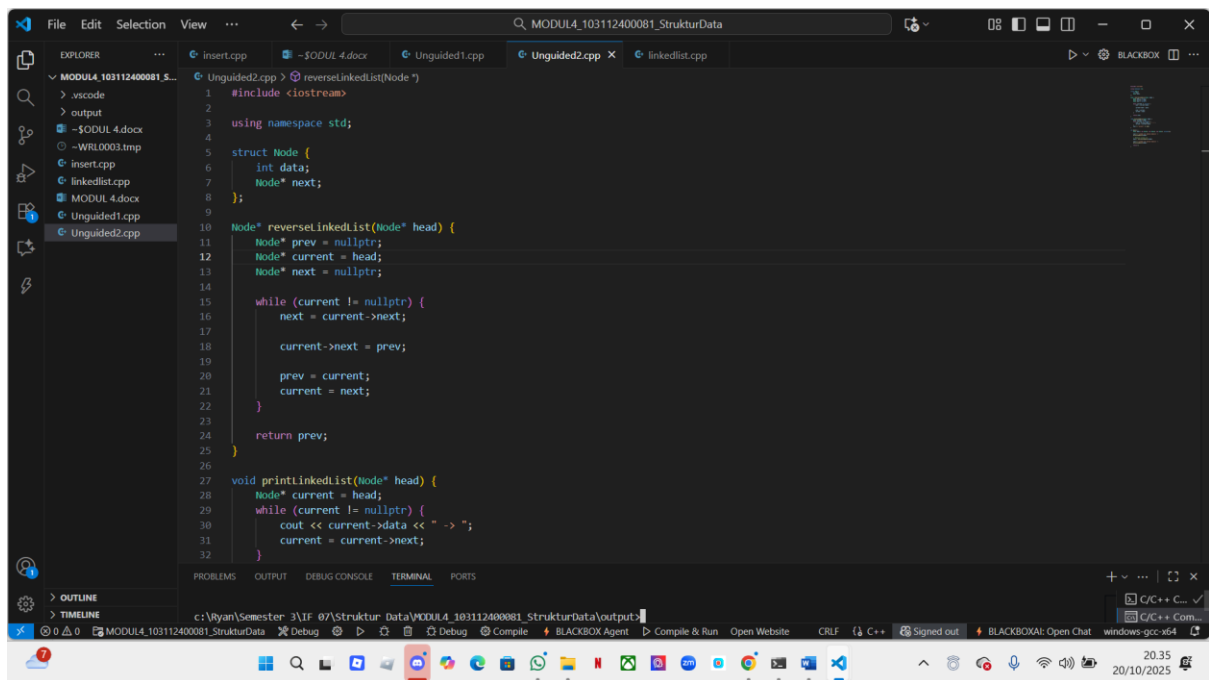
```
1 #include <iostream>
2 using namespace std;
3 struct Node {
4     int data;
5     Node* next;
6 };
7 class LinkedList {
8     Node* head;
9     Node* current;
10    void tambahAntrian() {
11        while (current != nullptr) {
12            cout << "Nama: " << current->data.name << ", Pesanan: " << current->data.pesanan << endl;
13            current = current->next;
14        }
15    }
16    int main() {
17        LinkedList antrian;
18        int pilihan;
19        string nama, pesanan;
20        while (true) {
21            cout << "Menu:" << endl;
22            cout << "1. Tambah Antrian" << endl;
23            cout << "2. Layani Antrian" << endl;
24            cout << "3. Tampilkan Antrian" << endl;
25            cout << "4. Keluar" << endl;
26            cout << "Masukkan pilihan Anda: ";
27            cin >> pilihan;
28            switch (pilihan) {
29                case 1:
30                    cout << "Masukkan nama pembeli: ";
31                    cin >> nama;
32                    cout << "Masukkan pesanan pembeli: ";
33                    cin >> pesanan;
34                    antrian.tambahAntrian(nama, pesanan);
35                    break;
36                case 2:
37                    antrian.layaniAntrian();
38                    break;
39                case 3:
40                    antrian.tampilkanAntrian();
41                    break;
42                case 4:
43                    cout << "Terima kasih!" << endl;
44                    return 0;
45                default:
46                    cout << "Pilihan tidak valid. Silakan coba lagi." << endl;
47            }
48        }
49    }
50    return 0;
51 }
```

The screenshot shows the Visual Studio Code editor with the same C++ file, `Unguided1.cpp`, but now the `main` function is being executed. The output window shows the program's execution flow, including the menu display and the user's input. The output is as follows:

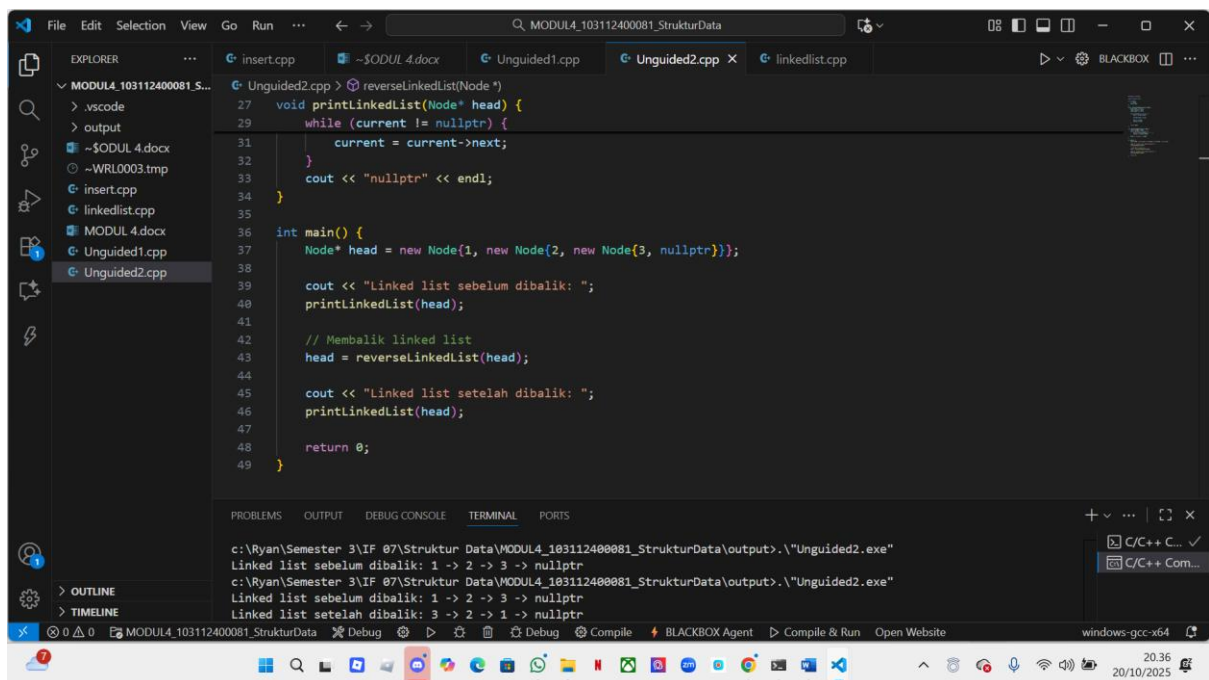
```
Antrian untuk Dean dengan pesanan Ayam telah ditambahkan.
Menu:
1. Tambah Antrian
2. Layani Antrian
3. Tampilkan Antrian
4. Keluar
Masukkan pilihan Anda: 3
Antrian:
Nama: Dea, Pesanan: Nasi
Nama: Dean, Pesanan: Ayam
Menu:
1. Tambah Antrian
2. Layani Antrian
3. Tampilkan Antrian
4. Keluar
Masukkan pilihan Anda: 
```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita menyimpan data pembeli(nama dan pesanan). Yang mana yang awal antri harus didahulukan.

Unguided 2



```
1 #include <iostream>
2
3 using namespace std;
4
5 struct Node {
6     int data;
7     Node* next;
8 };
9
10 Node* reverseLinkedList(Node* head) {
11     Node* prev = nullptr;
12     Node* current = head;
13     Node* next = nullptr;
14
15     while (current != nullptr) {
16         next = current->next;
17
18         current->next = prev;
19
20         prev = current;
21         current = next;
22     }
23
24     return prev;
25 }
26
27 void printLinkedList(Node* head) {
28     Node* current = head;
29     while (current != nullptr) {
30         cout << current->data << " -> ";
31         current = current->next;
32     }
```



```
27 void printLinkedList(Node* head) {
28     while (current != nullptr) {
29         current = current->next;
30     }
31     cout << "nullptr" << endl;
32 }
33
34 int main() {
35     Node* head = new Node{1, new Node{2, new Node{3, nullptr}}};
36
37     cout << "Linked list sebelum dibalik: ";
38     printLinkedList(head);
39
40     // Membalik linked list
41     head = reverseLinkedList(head);
42
43     cout << "Linked list setelah dibalik: ";
44     printLinkedList(head);
45
46     return 0;
47 }
```

Output:

```
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL4_103112400081_StrukturData\output>.\"Unguided2.exe"
Linked list sebelum dibalik: 1 -> 2 -> 3 -> nullptr
c:\Ryan\Semester 3\IF 07\Struktur Data\MODUL4_103112400081_StrukturData\output>.\"Unguided2.exe"
Linked list sebelum dibalik: 1 -> 2 -> 3 -> nullptr
Linked list setelah dibalik: 3 -> 2 -> 1 -> nullptr
```

Tujuan program ini adalah membuat program dari bahasa c++ yang mana kita memberikan implementasi praktis dari algoritma untuk membalik urutan node dalam sebuah singly linked list, yang merupakan operasi umum dalam struktur data dan algoritma.